

Homework #2

Name(s): Raj Shah (ras637), (), ()

Question 1: You are given an array $A[1..n]$ with the following properties:

- $A[1] > A[2]$ (the first element is greater than the second).
- $A[n-1] \leq A[n]$ (the second-to-last element is less than or equal to the last).

A local minimum is an element $A[x]$ such that $A[x-1] \geq A[x] \leq A[x+1]$. Design an $O(\log n)$ -time algorithm to find a local minimum.

Answer

Use binary search on the “slope”: check the middle index m . If $A[m]$ is not a local minimum, move to the half that contains a smaller neighbor, and continue until a local minimum is found.

Explanation

The array starts by decreasing and ends by increasing, so a local minimum must exist somewhere. If the middle element is greater than the left neighbor, then going left moves downhill, so a local minimum must exist in the left half. If it is greater than the right neighbor, then a local minimum must exist in the right half. Each step throws away half the array, which is why the time becomes logarithmic. A linear scan would also find a local minimum, but it can take $O(n)$ time.

Proof of Correctness

Claim: The algorithm returns an index x such that $A[x-1] \geq A[x] \leq A[x+1]$.

We use a loop invariant.

Loop Invariant: At the start of every iteration, the current interval $[l, r]$ contains at least one local minimum.

Initialization: Initially $[l, r] = [1, n]$. Since $A[1] > A[2]$ and $A[n-1] \leq A[n]$, the array starts with a “down step” and ends with an “up step”. Therefore, as we move from left to right, there must be an index where the direction changes from going down to not going down, which creates a local minimum. So the invariant holds.

Maintenance: Let $m = \lfloor (l + r)/2 \rfloor$. If $A[m]$ is a local minimum, the algorithm returns m and is correct. Otherwise, either $A[m] > A[m-1]$ or $A[m] > A[m+1]$. If $A[m] > A[m-1]$, then moving left from m goes downhill at first, and since the interval has a boundary and cannot decrease forever, a local minimum must exist in $[l, m-1]$; we set $r = m-1$. If $A[m] > A[m+1]$, a symmetric argument shows a local minimum exists in $[m+1, r]$; we set $l = m+1$. So the new interval still contains a local minimum, and the invariant is preserved.

Termination: Each update strictly shrinks the interval length, so the loop terminates. When it terminates by returning an index, that index satisfies the local-minimum condition by the check in the code. Therefore the returned index is correct.

Runtime Analysis

Each iteration does $O(1)$ work and halves the search interval.

$$T(n) = T(n/2) + O(1) \Rightarrow T(n) = O(\log n).$$

Pseudo code

Pseudo code

```
FindLocalMin(A[1..n]):  
    l = 1  
    r = n  
    while l <= r:  
        m = floor((l + r)/2)  
  
        leftOK = (m == 1) or (A[m-1] >= A[m])  
        rightOK = (m == n) or (A[m] <= A[m+1])  
        if leftOK and rightOK:  
            return m  
  
        if m > 1 and A[m] > A[m-1]:  
            r = m - 1  
        else:  
            l = m + 1
```

Final Answer (boxed)

Binary search on the slope finds a local minimum in $O(\log n)$ time.

Question 2: Suppose you are given two sorted arrays $A[1..n]$ and $B[1..n]$ containing distinct integers. Describe a $O(\log n)$ algorithm to find the median (meaning the n th smallest element) of the union $A \cup B$.

Answer

Binary search an index $i \in \{0, \dots, n\}$ such that the partition with i elements from A and $n - i$ elements from B is valid. When the partition is valid, the median is $\max(A[i], B[n - i])$ (with boundary values handled using $\pm\infty$).

Explanation

The median is the n th smallest among $2n$ numbers, so exactly n numbers should be on the left side of a correct partition. If we take i from A , we must take $n - i$ from B . The partition is correct when every element on the left is at most every element on the right, which can be checked using only the two border comparisons. If the left border from A is too big, we took too many from A and move left; otherwise we move right. Merging the arrays would take $O(n)$ time, which is slower than the required $O(\log n)$.

Proof of Correctness

Claim: The algorithm returns the n th smallest element of $A \cup B$.

Define a partition by choosing $i \in \{0, \dots, n\}$ and setting $j = n - i$. Let

$$A_L = \begin{cases} -\infty & i = 0 \\ A[i] & 1 \leq i \leq n \end{cases}, \quad A_R = \begin{cases} +\infty & i = n \\ A[i + 1] & 0 \leq i \leq n - 1 \end{cases}$$

and define B_L, B_R similarly using j . Call the partition *valid* if

$$A_L \leq B_R \quad \text{and} \quad B_L \leq A_R.$$

Why a valid partition gives the median: The left side contains exactly $i + j = n$ elements, and the inequalities ensure every left element is $\leq$ every right element. So the largest element on the left is exactly the n th smallest overall, which equals $\max(A_L, B_L)$.

Why binary search finds a valid partition: If $A_L > B_R$, then i is too large because the chosen left element from A is bigger than something that should be on the right; decreasing i moves A_L left and B_R right, fixing the problem. If $B_L > A_R$, then i is too small; increasing i moves A_R right and B_L left, fixing the problem. Because i ranges over a sorted search space and the direction of the fix is monotone, binary search must reach an i where neither inequality is violated, which is a valid partition. Therefore the returned $\max(A_L, B_L)$ is the median.

Runtime Analysis

Binary search over $i \in \{0, \dots, n\}$ does $O(1)$ work per step and takes $O(\log n)$ steps.

$$\boxed{O(\log n)}.$$

Pseudo code

Pseudo code

```
MedianTwoSorted(A[1..n], B[1..n]):  
    lo = 0  
    hi = n  
    while lo <= hi:  
        i = floor((lo + hi)/2)  
        j = n - i  
  
        Aleft  = -infty if i==0 else A[i]  
        Aright = +infty if i==n else A[i+1]  
        Bleft  = -infty if j==0 else B[j]  
        Bright = +infty if j==n else B[j+1]  
  
        if Aleft <= Bright and Bleft <= Aright:  
            return max(Aleft, Bleft)  
        else if Aleft > Bright:  
            hi = i - 1  
        else:  
            lo = i + 1
```

Final Answer (boxed)

Binary search a valid partition and output $\max(A[i], B[n - i])$ in $O(\log n)$ time.

Question 3: Write an algorithm in pseudocode:

- **Input:** An array A with n distinct (non-equal) elements
- **Output-1:** numbers x and y in A that minimize $|x - y|$
- **Output-2:** a pair of numbers w and z in A that minimize $|w + z|$

The run-time should be significantly better than $O(n^2)$.

Answer

Sort A . For Output-1, the closest pair must be adjacent in sorted order, so scan adjacent pairs. For Output-2, use a two-pointer scan from both ends to minimize $|w + z|$.

Explanation

After sorting, the smallest difference $|x - y|$ must occur between neighbors because any number between them would create a smaller gap. For minimizing $|w + z|$, start with the smallest and largest values and look at the sum. If the sum is negative, increase the left pointer to make the sum larger; if positive, decrease the right pointer to make the sum smaller. This is correct because the array is sorted and pointer moves change the sum in a predictable direction. Brute force checks all pairs and costs $O(n^2)$, which is too slow.

Proof of Correctness

Claim 1: The algorithm for Output-1 returns a pair (x, y) minimizing $|x - y|$.

Let A' be the sorted array. Consider an optimal pair $A'[p] < A'[q]$ with $q > p$. If $q > p + 1$, then $A'[p] < A'[p + 1] < A'[q]$, so

$$A'[p + 1] - A'[p] < A'[q] - A'[p],$$

contradicting optimality of (p, q) . Therefore $q = p + 1$, so an optimal pair is adjacent, and the adjacent scan finds it.

Claim 2: The two-pointer algorithm for Output-2 returns a pair (w, z) minimizing $|w + z|$.

Loop Invariant: At each step with pointers (i, j) , all pairs outside the current window cannot produce a smaller absolute sum than the best seen so far.

Maintenance: If $A'[i] + A'[j] < 0$, then for fixed j , decreasing i would only make the sum smaller (more negative), so it cannot reduce $|\cdot|$ toward 0. Thus moving i right is the only move that can improve closeness to 0. If $A'[i] + A'[j] > 0$, then for fixed i , increasing j would only make the sum larger (more positive), so moving j left is the only move that can improve closeness to 0. Therefore the algorithm never discards a pair that could beat the best answer, and it eventually checks a best pair.

Runtime Analysis

Sorting takes $O(n \log n)$. The adjacent scan is $O(n)$ and the two-pointer scan is $O(n)$.

$$\boxed{O(n \log n)}.$$

Pseudo code

Pseudo code

```
ClosestPairs(A[1..n]):
    sort A increasing

    # Output-1: minimize |x - y|
    bestDiff = +infty
    x = A[1]; y = A[2]
    for k = 1 to n-1:
        d = A[k+1] - A[k]
        if d < bestDiff:
            bestDiff = d
            x = A[k]; y = A[k+1]

    # Output-2: minimize |w + z|
    i = 1
    j = n
    bestAbs = +infty
    w = A[i]; z = A[j]
    while i < j:
        s = A[i] + A[j]
        if abs(s) < bestAbs:
            bestAbs = abs(s)
            w = A[i]; z = A[j]
        if s < 0:
            i = i + 1
        else:
            j = j - 1

    return (x, y, w, z)
```

Final Answer (boxed)

Sort then scan neighbors and two pointers; total time $O(n \log n)$.

Question 4: An inversion (swap) in an array $A[1..n]$ is a pair of indices (i, j) such that $i < j$ and $A[i] > A[j]$. Describe and analyze an algorithm to count the number of inversions in an n -element array in $O(n \log n)$ time.

Answer

Modify mergesort: recursively count inversions in the left half and right half, and count split inversions during the merge step. Add these three counts.

Explanation

Every inversion either stays completely inside the left half, inside the right half, or it crosses from left to right. The recursion counts the inversions inside each half. During merge, both halves are sorted, so when a right element is smaller than the current left element, it is also smaller than all remaining left elements. That lets us count many inversions at once instead of checking pairs. A brute force method checks all pairs and takes $O(n^2)$ time.

Proof of Correctness

Claim: The algorithm returns the exact number of inversions in $A[1..n]$.

We prove by induction on the subarray length $k = r - l + 1$ for the function `SortCount(A, l, r)`.

Base Case: If $k \leq 1$, there are no pairs (i, j) with $i < j$, so the inversion count is 0. The algorithm returns 0.

Inductive Hypothesis: Assume for all arrays of length $< k$, the algorithm returns the correct inversion count.

Inductive Step: For length k , split at m . By the inductive hypothesis, the recursive calls return the correct number of inversions entirely in the left half and entirely in the right half. It remains to show `MergeCount` returns exactly the number of split inversions. During merge, when we take $R[j]$ before $L[i]$, we know $R[j] < L[i]$, and since L is sorted, $R[j]$ is smaller than every element $L[i], L[i + 1], \dots, L[|L|]$. This creates exactly $|L| - i + 1$ split inversions, and no other step creates split inversions. Therefore the merge count is correct, so the total sum is correct.

Runtime Analysis

Mergesort does two recursive calls on size $n/2$ and the merge step is $O(n)$:

$$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n).$$

Pseudo code

Pseudo code

```
CountInv(A[1..n]):
    return SortCount(A, 1, n)

SortCount(A, l, r):
    if l >= r: return 0
    m = floor((l + r)/2)
    c1 = SortCount(A, l, m)
    c2 = SortCount(A, m+1, r)
    c3 = MergeCount(A, l, m, r)
    return c1 + c2 + c3

MergeCount(A, l, m, r):
    L = A[l..m], R = A[m+1..r]
    i = 1; j = 1; k = l
    inv = 0
    while i <= len(L) and j <= len(R):
        if L[i] <= R[j]:
            A[k] = L[i]; i = i + 1
        else:
            A[k] = R[j]; j = j + 1
            inv = inv + (len(L) - i + 1)
        k = k + 1
    copy remaining of L and R into A
    return inv
```

Final Answer (boxed)

Count inversions via modified mergesort in $O(n \log n)$ time.
--

Question 5(a): Given an array A of length n where all elements are positive (may include fractions), output the maximum value of $\text{Prod}(A, i, j)$ over all subarrays $A[i..j]$. Write pseudocode for a recursive algorithm that solves the problem in $O(n \log n)$ time. Provide the pseudocode and the recurrence relation.

Answer

Use divide-and-conquer. Recursively compute the best product subarray in the left half and right half, and compute the best product subarray that crosses the midpoint by multiplying the best suffix product of the left side with the best prefix product of the right side.

Explanation

The best subarray either lies fully in the left half, fully in the right half, or crosses the midpoint. The recursive calls handle the first two cases. For the crossing case, since all numbers are positive, the best crossing product is the maximum suffix product ending at the midpoint times the maximum prefix product starting after it. Those two values can be found with simple linear scans from the middle outward. A brute force method checks all subarrays and takes $O(n^2)$ time.

Proof of Correctness

Claim: $\text{MaxProdRec}(A, l, r)$ returns the maximum product of any subarray of $A[l..r]$.

We prove by induction on $k = r - l + 1$.

Base Case: If $k = 1$, the only subarray is $[l, l]$, and the algorithm returns $A[l]$, which is correct.

Inductive Hypothesis: Assume the claim holds for all segments of length $< k$.

Inductive Step: Split $[l, r]$ at m . By the inductive hypothesis, **leftBest** is the best product entirely in $[l, m]$, and **rightBest** is the best product entirely in $[m + 1, r]$. Any subarray that crosses m must be a suffix of $[l, m]$ times a prefix of $[m + 1, r]$. Because all entries are positive, maximizing the product is done by independently maximizing the suffix product and the prefix product, so the crossing best equals **bestSuffix**·**bestPrefix**. Taking the maximum of the three cases covers all possible subarrays, so the result is correct.

Runtime Analysis

The combine work is two linear scans, so it is $O(n)$ at each level:

$$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n).$$

Pseudo code

Pseudo code

```
MaxProd(A[1..n]):
    return MaxProdRec(A, 1, n)

MaxProdRec(A, l, r):
    if l == r: return A[l]
    m = floor((l + r)/2)

    leftBest  = MaxProdRec(A, l, m)
    rightBest = MaxProdRec(A, m+1, r)

    bestSuffix = 0
    p = 1
    for i = m downto l:
        p = p * A[i]
        if p > bestSuffix: bestSuffix = p

    bestPrefix = 0
    p = 1
    for j = m+1 to r:
        p = p * A[j]
        if p > bestPrefix: bestPrefix = p

    crossBest = bestSuffix * bestPrefix
    return max(leftBest, rightBest, crossBest)
```

Final Answer (boxed)

$$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n).$$

Question 5(b): Now, write pseudocode for a recursive algorithm that solves the problem in $O(n)$ time. Provide the pseudocode and the recurrence relation.

Answer

Return a 4-tuple for each segment: total product, best prefix product, best suffix product, and best subarray product. Combine two halves in $O(1)$ time using these values, giving an $O(n)$ algorithm.

Explanation

The maximum-product subarray for a segment is either in the left half, in the right half, or it crosses the middle. A crossing subarray is the best suffix of the left half times the best prefix of the right half. If we store total product, best prefix, best suffix, and best subarray for each half, we can compute the same four values for the combined segment using constant-time formulas. This is

similar to how max-sum subarray can be done with divide-and-conquer summaries. The $O(n \log n)$ approach is slower because it scans $O(n)$ at every recursion level.

Proof of Correctness

Claim: For any segment $A[l..r]$, $\text{MaxProdX}(A, l, r)$ returns a tuple

$$(\text{total}, \text{bestPrefix}, \text{bestSuffix}, \text{bestSubarray})$$

that is correct for $A[l..r]$.

We prove by induction on $k = r - l + 1$.

Base Case: If $k = 1$, total, prefix, suffix, and best subarray are all $A[l]$, and the function returns $(A[l], A[l], A[l], A[l])$, which is correct.

Inductive Hypothesis: Assume the tuple is correct for all segments of length $< k$.

Inductive Step: Split at m . By the hypothesis, the left tuple and right tuple are correct. The total product is $\text{tL} \cdot \text{tR}$. The best prefix is either the left best prefix, or the entire left total times the right best prefix, so $\text{bestPrefix} = \max(\text{preL}, \text{tL} \cdot \text{preR})$. Similarly, $\text{bestSuffix} = \max(\text{sufR}, \text{tR} \cdot \text{sufL})$. Any best subarray is either in left, in right, or crossing, and the best crossing product is $\text{sufL} \cdot \text{preR}$, so bestSub is the maximum of these three. Therefore the combined tuple is correct.

Runtime Analysis

The combine step is $O(1)$ and there are two recursive calls:

$$T(n) = 2T(n/2) + O(1).$$

At recursion level h there are 2^h subproblems and total combine work $O(2^h)$. Summing over $h = 0$ to $\log n$ gives $O(n)$.

$$\boxed{T(n) = O(n)}.$$

Pseudo code

Pseudo code

```
# Returns (totalProd, bestPrefix, bestSuffix, bestSubarray)
MaxProdX(A, l, r):
    if l == r:
        v = A[l]
        return (v, v, v, v)

    m = floor((l + r)/2)
    (tL, preL, sufL, bestL) = MaxProdX(A, l, m)
    (tR, preR, sufR, bestR) = MaxProdX(A, m+1, r)

    total = tL * tR
    bestPrefix = max(preL, tL * preR)
    bestSuffix = max(sufR, tR * sufL)
    crossBest = sufL * preR
    bestSub = max(bestL, bestR, crossBest)

    return (total, bestPrefix, bestSuffix, bestSub)

MaxProd(A[1..n]):
    (_, _, _, ans) = MaxProdX(A, 1, n)
    return ans
```

Final Answer (boxed)

$$T(n) = 2T(n/2) + O(1) \Rightarrow T(n) = O(n).$$

Question 6(a): You are given two strings A and B of length n , over an alphabet of size M . Design an algorithm that decides if they are anagrams with worst-case runtime $O(n + M)$.

Answer

Use a counting array $C[1..M]$ initialized to 0. Scan A and increment counts, scan B and decrement counts, and finally check that all entries of C are 0.

Explanation

Two strings are anagrams exactly when every character appears the same number of times in both strings. A size- M array lets us update the count of a character in $O(1)$ time by indexing. After incrementing for A and decrementing for B , any nonzero entry means the strings differ in frequency of that character. This uses $O(n)$ time for the scans and $O(M)$ time for the final check. Sorting is slower here because it takes $O(n \log n)$ time.

Proof of Correctness

Claim: The algorithm returns **true** iff A and B are anagrams.

If the algorithm returns **true**, then for every character c , $C[c] = 0$. That means the number of occurrences of c in A equals the number of occurrences of c in B . So the multisets of characters match and B is a reordering of A , meaning they are anagrams.

If A and B are anagrams, then every character c appears the same number of times in both. The algorithm increments $C[c]$ once per occurrence in A and decrements $C[c]$ once per occurrence in B . Therefore all counts end at 0, so the final check passes and the algorithm returns **true**. Hence the algorithm is correct.

Runtime Analysis

Initializing C takes $O(M)$, scanning both strings takes $O(n)$, and checking all counts takes $O(M)$.

$$O(n + M).$$

Pseudo code

Pseudo code

```
IsAnagram_Count(A[1..n], B[1..n], M):
    C[1..M] = all zeros
    for i = 1 to n:
        C[code(A[i])] = C[code(A[i])] + 1
    for i = 1 to n:
        C[code(B[i])] = C[code(B[i])] - 1
    for c = 1 to M:
        if C[c] != 0:
            return false
    return true
```

Final Answer (boxed)

Count characters with an array of size M ; time $O(n + M)$.

Question 6(b): Design a randomized algorithm for this problem with worst-case expected runtime $O(n)$.

Answer

Pick a large prime P and random integers $a \in \{1, \dots, P-1\}$ and $b \in \{0, \dots, P-1\}$. Define a hash $h(c) = (a \cdot c + b) \bmod P$. Compute $S_A = \sum h(\text{code}(A[i])) \bmod P$ and $S_B = \sum h(\text{code}(B[i])) \bmod P$; return **true** iff $S_A = S_B$.

Explanation

If two strings are anagrams, they contain the same multiset of characters, so the hashed sum is the same. If they are not anagrams, the two multisets differ, so the difference of sums is a nonzero expression in the hash values. Because a and b are chosen randomly, this difference equals 0 modulo P only with small probability. The algorithm uses only one scan of each string, so it runs in $O(n)$ time. The counting method from part (a) is slower when M is much larger than n because it needs $O(M)$ work.

Proof of Correctness

Claim: If the algorithm returns **false**, then A and B are not anagrams. If it returns **true**, then A and B are anagrams with high probability.

If A and B are anagrams, then they have the same characters with the same multiplicities. Therefore the multiset of hashed values is the same, so $S_A = S_B$ always, and the algorithm always returns **true**.

If A and B are not anagrams, then there exists at least one character whose frequency differs, so the difference

$$D = \left(\sum h(\text{code}(A[i])) - \sum h(\text{code}(B[i])) \right) \bmod P$$

is not identically zero as a function of (a, b) . For randomly chosen (a, b) , the probability that $D = 0$ modulo P is at most $1/P$ (so false positives are rare). Therefore the algorithm is a Monte Carlo test: it never misses a true anagram, and it only rarely accepts a non-anagram.

Runtime Analysis

Each hash evaluation is $O(1)$ and we compute two sums over n characters.

$O(n)$ time (expected and worst-case).

Pseudo code

Pseudo code

```
IsAnagram_Random(A[1..n], B[1..n]):  
    pick a large prime P  
    pick random a in {1,...,P-1}  
    pick random b in {0,...,P-1}  
  
    sA = 0  
    sB = 0  
    for i = 1 to n:  
        sA = (sA + (a*code(A[i]) + b) mod P) mod P  
        sB = (sB + (a*code(B[i]) + b) mod P) mod P  
  
    return (sA == sB)
```

Final Answer (boxed)

Random linear hash sum modulo P gives expected $O(n)$ time with small error.

Question 7(a): Suppose we sample an index i from $\{1, \dots, n\}$ uniformly at random. What is the probability that i is a correct answer, i.e., $A[i] \bmod 5 = 0$?

Answer

The probability is $\frac{1}{5}$.

Explanation

Because n is a multiple of 5, exactly $n/5$ numbers in $\{1, \dots, n\}$ are divisible by 5. Since A is a permutation, those $n/5$ values appear in exactly $n/5$ positions in the array. A uniformly random index hits one of those positions with probability $(n/5)/n$. The order of the permutation does not matter for this probability. Therefore the probability is $1/5$.

Proof of Correctness

Claim: The probability is $\frac{1}{5}$.

There are exactly $n/5$ valid indices because there are exactly $n/5$ multiples of 5 in the set and each appears once in the permutation. A uniform random choice from n indices has probability $(n/5)/n = 1/5$ to land on one of these. Hence the probability is $\frac{1}{5}$.

Runtime Analysis

This is a probability computation, so runtime is not applicable.

Final Answer (boxed)

$$\boxed{\frac{1}{5}}$$

Question 7(b): Suppose we sample m indices from $\{1, \dots, n\}$ uniformly at random and with repetition. What is the probability that none of these indices is a correct answer?

Answer

The probability is $\left(\frac{4}{5}\right)^m$.

Explanation

Each independent sample succeeds with probability $1/5$ and fails with probability $4/5$. Because sampling is with repetition, the samples are independent. The probability that all m samples fail is the product of $4/5$ taken m times. This matches the standard rule for repeated independent trials. So the probability is $(4/5)^m$.

Proof of Correctness

Claim: The probability that none of the m samples is correct equals $(4/5)^m$.

For each sample, the event “index is not correct” has probability $4/5$. By independence, the probability that all m samples are not correct is

$$\left(\frac{4}{5}\right) \cdot \left(\frac{4}{5}\right) \cdots \left(\frac{4}{5}\right) = \left(\frac{4}{5}\right)^m.$$

Hence the expression is correct.

Runtime Analysis

This is a probability computation, so runtime is not applicable.

Final Answer (boxed)

$$\boxed{\left(\frac{4}{5}\right)^m}$$

Question 7(c): What is the worst-case expected running time of **Find-Index-1**($A[1 : n]$)? Remember to prove your answer formally.

Answer

The worst-case expected number of iterations is 5, so the worst-case expected running time is $O(1)$.

Explanation

Each iteration samples a uniformly random index and checks divisibility by 5. In any permutation, exactly $n/5$ positions are correct, so each iteration succeeds with probability $1/5$. This makes the number of iterations a geometric random variable with parameter $p = 1/5$. The expected number of trials for a geometric variable is $1/p = 5$. Therefore the expected running time is constant.

Proof of Correctness

Claim: The expected number of iterations of **Find-Index-1** is 5.

In every iteration, the algorithm samples i uniformly at random, and $\Pr[A[i] \bmod 5 = 0] = 1/5$. Let T be the number of trials until the first success. Then T is geometric with parameter $p = 1/5$, so

$$\mathbb{E}[T] = \frac{1}{p} = 5.$$

This probability does not depend on the input permutation, so it is worst-case as well.

Runtime Analysis

Each iteration does $O(1)$ work, and $\mathbb{E}[T] = 5$, so

$$\boxed{\mathbb{E}[\text{time}] = O(1).}$$

Final Answer (boxed)

$$\boxed{\mathbb{E}[\text{running time}] = O(1) \text{ (specifically, 5 expected iterations).}}$$

Question 7(d): What is the worst-case running time of `Find-Index-2`($A[1 : n]$)? What about its worst-case expected running time? Remember to prove your answer formally.

Answer

Worst-case running time is $O(n)$. Worst-case expected running time is $O(1)$.

Explanation

In the worst case, the algorithm does n random samples and then scans the full array, which is at most $2n$ checks. That is why the worst-case time is $O(n)$. The probability that all n random samples fail is $(4/5)^n$. The scan only happens in that rare event, so its expected cost is $n(4/5)^n$. The expected number of random samples before success is at most 5, so the total expected time is constant.

Proof of Correctness

Claim: `Find-Index-2` always returns a correct index, and its worst-case expected time is $O(1)$.

Correctness is guaranteed because if the randomized loop finds $A[i] \bmod 5 = 0$, it returns a correct index. If it never finds one in n trials, the algorithm performs a deterministic scan, and since the array contains all numbers $1..n$, it must contain multiples of 5, so the scan will find and return a correct index.

For expected time, let F be the event that all n random samples fail. Then

$$\Pr[F] = \left(\frac{4}{5}\right)^n.$$

The scan costs $O(n)$ and happens only when F occurs, so its expected cost is $O(n(4/5)^n)$. The randomized part is a geometric process with success probability $1/5$, truncated at n , so the expected number of random trials is at most 5. Therefore the total expected time is bounded by a constant plus $n(4/5)^n$, which is $O(1)$.

Runtime Analysis

Worst-case time: n random checks + n scan checks $\leq 2n$, so $O(n)$. Expected time:

$$\mathbb{E}[\text{time}] \leq O(1) + O\left(n \left(\frac{4}{5}\right)^n\right) = O(1).$$

Final Answer (boxed)

Worst-case time $O(n)$ and worst-case expected time $O(1)$.

Question 8(1): Challenge Yourself. If all moves must be between adjacent pegs (so you cannot directly move from peg 1 to peg 3), what is the number of moves required?

Answer

Let $T(n)$ be the minimum number of moves with adjacent-only moves. Then

$$T(1) = 2, \quad T(n) = 3T(n-1) + 2,$$

so $T(n) = 3^n - 1$.

Explanation

With adjacent-only moves, the largest disk must move from peg 1 to peg 3 by going 1 to 2 and then 2 to 3, which takes 2 moves. Before each of those two moves, all $n-1$ smaller disks must be moved out of the way. That creates three subproblems of size $n-1$ total: clear for the first move, clear for the second move, and finally stack the small disks back on top. This gives $T(n) = 3T(n-1) + 2$. Solving the recurrence gives $3^n - 1$ moves.

Proof of Correctness

Claim: The recurrence $T(n) = 3T(n-1) + 2$ correctly describes the minimum number of moves, and its solution is $T(n) = 3^n - 1$.

To move the largest disk from peg 1 to peg 3, adjacent-only rules force two moves: $1 \rightarrow 2$ then $2 \rightarrow 3$. Before $1 \rightarrow 2$, all $n-1$ smaller disks must be moved from peg 1 to peg 3, costing $T(n-1)$. Before $2 \rightarrow 3$, those $n-1$ disks must be moved from peg 3 to peg 1 (so peg 3 is free), costing another $T(n-1)$. After moving the largest disk to peg 3, the $n-1$ disks must be moved from peg 1 to peg 3, costing a third $T(n-1)$. Thus any valid solution needs at least $3T(n-1) + 2$ moves, and this strategy achieves it, so it is optimal.

Runtime Analysis

Solve:

$$T(n) = 3T(n-1) + 2, \quad T(1) = 2.$$

Unrolling gives:

$$T(n) = 3^n - 1.$$

Final Answer (boxed)

$T(n) = 3^n - 1$

Question 8(2a): Challenge Yourself. For each disk size k , there are two identical disks. If we ignore identicalness and treat them as distinct disks, apply standard Tower of Hanoi to $2n$ disks. What is the recurrence relation and the resulting number of moves?

Answer

For m distinct disks, standard Hanoi satisfies $T(m) = 2T(m-1) + 1$ with $T(1) = 1$. Here $m = 2n$, so the number of moves is $2^{2n} - 1 = 4^n - 1$.

Explanation

Standard Hanoi with m distinct disks works by moving the top $m - 1$ disks aside, moving the largest disk once, then moving the $m - 1$ disks onto it. That gives the recurrence $T(m) = 2T(m - 1) + 1$. Solving gives $T(m) = 2^m - 1$. If we treat the two identical copies as distinct, then there are $2n$ disks total. Substituting $m = 2n$ gives $2^{2n} - 1 = 4^n - 1$ moves.

Proof of Correctness

Claim: Treating the disks as distinct reduces to standard Hanoi on $2n$ disks, requiring $2^{2n} - 1$ moves.

The standard Hanoi recurrence is correct because any valid solution must move the top $m - 1$ disks off the largest disk before moving it, then must move those $m - 1$ disks onto the target peg afterward. The recurrence counts exactly those necessary steps and the known algorithm achieves it, so it is optimal. Setting $m = 2n$ directly gives the move count $2^{2n} - 1$.

Runtime Analysis

$$T(1) = 1, \quad T(m) = 2T(m - 1) + 1 \Rightarrow T(m) = 2^m - 1, \quad m = 2n.$$

Final Answer (boxed)

$$\boxed{2^{2n} - 1 = 4^n - 1}$$

Question 8(2b): Challenge Yourself. If we move both disks of the same size simultaneously while using standard Hanoi's algorithm, this may flip the order of equal-size disks. Repeat this again to restore the order. How many moves does this take?

Answer

Treat each pair as one block and run standard Hanoi on n blocks, which takes $2^n - 1$ moves, then repeat once more. Total moves: $2(2^n - 1) = 2^{n+1} - 2$.

Explanation

If each identical pair is moved together, it behaves like a single larger unit, so the process matches standard Hanoi on n units. That takes $2^n - 1$ moves. Because the two copies are identical, the first pass can reverse their internal order within a pair. Repeating the same process applies the same flip again, restoring the original order. So the total number of moves is twice the standard count.

Proof of Correctness

Claim: The described strategy uses $2(2^n - 1)$ moves and preserves the final order.

One pass treats each pair as a block and moves all blocks from peg 1 to peg 3 in $2^n - 1$ moves. Any order change within identical pairs comes from the sequence of block moves. Running the same block sequence again applies the same internal permutation a second time, which returns every pair to its original order. Therefore after two passes, the blocks are on peg 3 and the original order within each pair is restored, using exactly $2(2^n - 1)$ moves.

Runtime Analysis

Two standard Hanoi runs on n blocks:

$$2(2^n - 1) = 2^{n+1} - 2.$$

Final Answer (boxed)

$$\boxed{2(2^n - 1) = 2^{n+1} - 2}$$

Question 8(2c): Challenge Yourself. What is the optimal strategy to minimize the number of moves while preserving the original order of the disks?

Answer

Let $T(n)$ be the minimum moves to move n sizes (two disks per size) while preserving the original order within each size. Then

$$T(1) = 3, \quad T(n) = 2T(n-1) + 3,$$

so $T(n) = 3 \cdot 2^n - 3$.

Explanation

To keep order, the two largest equal disks cannot move as a single block because the top one must end above the bottom one in the same order. Moving the largest pair from peg 1 to peg 3 needs exactly 3 moves: top copy to auxiliary, bottom copy to target, then top copy onto it. Before doing this, all smaller disks must be moved away, and after, moved back on top, which costs $2T(n-1)$. This gives $T(n) = 2T(n-1) + 3$. Solving gives $3 \cdot 2^n - 3$ moves.

Proof of Correctness

Claim: $T(n) = 2T(n-1) + 3$ is optimal, and $T(n) = 3 \cdot 2^n - 3$.

Lower bound: To move the largest size pair while preserving their order, the top copy must temporarily leave the stack, then the bottom copy moves to the target, then the top copy moves onto it. This requires at least 3 moves involving the largest size disks. Also, before the bottom copy can move, all smaller disks must be removed from above it, and after placing the largest pair, all smaller disks must be moved onto the target, requiring at least $T(n-1)$ moves before and $T(n-1)$ moves after. So any algorithm needs at least $2T(n-1) + 3$ moves.

Achievability: Move the $n-1$ smaller sizes from peg 1 to peg 2 (preserving order) using $T(n-1)$ moves. Perform the 3 moves to transfer the largest pair from peg 1 to peg 3 while preserving order. Then move the $n-1$ smaller sizes from peg 2 to peg 3 using another $T(n-1)$ moves. This uses exactly $2T(n-1) + 3$ moves, matching the lower bound, so it is optimal.

Runtime Analysis

Solve:

$$T(1) = 3, \quad T(n) = 2T(n-1) + 3.$$

Unrolling:

$$T(n) = 3 \cdot 2^n - 3.$$

Final Answer (boxed)

$$T(n) = 2T(n-1) + 3, T(1) = 3 \Rightarrow T(n) = 3 \cdot 2^n - 3$$